Task D 学习总结

把 RAG 链路装进 FastAPI：POST /ask

2026-10-12 · Day 6 · P1 收尾

RAG 问答 API：检索 → 精排 → 生成，完整走通 HTTP

这份总结记录 Task D 的完整实现：把验证过的 rag_qa.py 链路包装成 **ask_rag(question)**，再用 FastAPI 暴露为 **POST /ask**。包含 5 个关键设计决策、7 个真实踩坑（含 2 个假警报的鉴别），以及 5 道仿照真实面试题改编的 Task D 定制面试题（中文思路 + English 答案）。

第一部分：要点总结

1. 做了什么

把 rag_qa.py 里验证过的 RAG 链路——Chroma 检索 top-10 → bge-reranker-v2-m3 精排取 top-3 → 拼接 graded-honesty prompt → Gemini 生成——包成函数 **ask_rag(question) -> (answer, sources)**，再用 FastAPI 暴露为 **POST /ask**。请求体为 {"question": "..."}，返回 {"answer": "...", "sources": [{"text": ..., "distance": ...}]}。用 **uvicorn app:app --reload** 启动，浏览器打开 **/docs** 发请求验证通过。

```
class AskRequest(BaseModel):
    question: str

class AskResponse(BaseModel):
    answer: str
    sources: list

@app.post("/ask")
def ask(req: AskRequest) -> AskResponse:
    answer, sources = ask_rag(req.question)
    return AskResponse(answer=answer, sources=sources)
```

2. 关键设计决策

决策 1：重的东西只加载一次。 Chroma PersistentClient、embedding 函数、CrossEncoder reranker、genai.Client() 全部放在模块顶层，import 时执行一次；每次请求只跑“查 → 排 → 拼 → 生成”。这是 build-slow / query-fast 的两段式，和 ingest
阶段的思路一致：启动慢一次，查询次次快。

决策 2：Pydantic 定契约。 AskRequest / AskResponse
做类型声明：请求体缺字段或类型不对，FastAPI 直接返回 422，handler 里拿到的 req.question 一定是 str。同时 OpenAPI 文档和 /docs 交互式调试页自动生成，前端不用等后端手写文档。

决策 3：POST + JSON body，不用 GET。 question 是任意长的自然语言：GET 的 URL
有长度限制、需要编码、还会进服务器访问日志；POST body 语义也对——这是一次“计算 / 生成”操作，不是读取静态资源。以后加 top_k、temperature 等参数，JSON body
直接加字段，GET 则要一个个拼 query param。

决策 4：graded-honesty prompt 在 API 层稳定复现。 /docs 实测 "What is
RAG?"：能回答的部分结构化作答，资料没覆盖的（RAG 全称）明确标注 "Not covered by the context"，没有幻觉。这是 Day 3 调出来的 over-refusal / hallucination seesaw 平衡点，现在在 HTTP 接口里稳定复现。

决策 5：附带验证了 Day 2 的结论。 返回的 3 个 source 按 reranker
分数排序：余弦距离最近（0.375）的 chunk 只排第三，定义性
chunk（0.414）排第一。cross-encoder
按语义相关性重排，不按向量距离；两种分数量纲不同、不可混读——Day 2 的“分数只在同一 reranker 内可比”在这里又用上一次。

第二部分：踩坑记录（7 个，含 2 个假警报）

坑 1：results["distance"] → KeyError

现象：Chroma 的 collection.query() 返回字典里，距离字段叫 "distances"（复数），写成单数直接 KeyError。

教训：SDK 返回的字段名不要凭记忆写：打印一次 keys 或查文档确认。

坑 2：PROMPT_TEMPLATE.format(question, context) → KeyError: 'context'

现象：template 里是命名占位符 {question} / {context}，调用时却传了位置参数，Python 按名字查找失败。

教训：规则：{} 用位置参数传，{名字} 必须用 名字=值 传。混用就炸。

坑 3：genai.GenerativeModel(...) → AttributeError

现象：新旧 SDK 混用：from google import genai 是新版 SDK，根本没有 GenerativeModel（那是旧版 google-generativeai 的类）。

教训：import 语句决定了你能用的 API 形状，换 SDK 要整套换。新版写法：client = genai.Client()（自动读取 .env 里的 GEMINI_API_KEY）+ client.models.generate_content(model=..., contents=...)。

```
from google import genai
from google.genai import types

genai_client = genai.Client()  # auto-reads GEMINI_API_KEY
response = genai_client.models.generate_content(
    model="gemini-3.5-flash",
    contents=prompt,
    config=types.GenerateContentConfig(max_output_tokens=2048),
)
```

坑 4：“程序没有运行”假象

现象：if __name__ == "__main__": 里调用了 ask_rag("What is RAG?") 但没接返回值、没 print。程序其实全跑完了：日志里两个模型加载 100%、generate_content 已调用、全程无 traceback——只是屏幕没输出。

教训：先看日志证据再下“没跑起来”的结论：loading 完成 + 无 traceback = 跑完了，只是没打印。

坑 5：source“被截断”假警报

现象：调试打印写了 s["text"][:120]，只取前 120 字符显示；sources 里的数据本身是完整的，API 返回也是全量。

教训：区分“显示截断”和“数据截断”：先看数据本身，再怀疑链路。

坑 6：答案说到半句停

现象：某次运行答案在半句结束。第一诊断位是 response.candidates[0].finish_reason：MAX_TOKENS = 撞到输出 token 上限，用 GenerateContentConfig(max_output_tokens=...) 调大；STOP =

模型自己正常结束——本次是生成随机性导致的偶发提前收尾，重跑即完整。

教训：finish_reason 是第一诊断位，别靠猜。只有反复半句停且为 STOP 时，才需要在 prompt 里加“请完整回答”。

坑 7：别把 reranker 分数当余弦距离读

现象：sources 里打印的 distance 是 Chroma 的余弦距离，排序依据却是 bge 的 cross-encoder 分数。两者量纲不同、不可比，混读会得出“排序错了”的错误结论。

教训：Day 2 结论的实战应用：跨模型的分数没有可比性，读数之前先确认这个数是谁打的。

Part 3: Detailed Version (English)

What was built

Task D wrapped the validated RAG pipeline from rag_qa.py — Chroma top-10 retrieval, bge-reranker-v2-m3 cross-encoder reranking to top-3, graded-honesty prompt assembly, Gemini generation — into a function **ask_rag(question) -> (answer, sources)**, exposed via FastAPI as **POST /ask**. Request body: {"question": "..."}; response: {"answer": "...", "sources": [{"text": ..., "distance": ...}]}. Served with **uvicorn app:app --reload**, verified through the auto-generated **/docs** page.

Key design decisions

1. Load heavy objects once. The Chroma PersistentClient, the embedding function, the CrossEncoder reranker, and genai.Client() all live at module top level and execute once at import time. Each request only runs retrieve → rerank → assemble → generate. This is the build-slow / query-fast two-stage pattern, consistent with the ingest stage: pay startup cost once, keep every query fast.

2. Pydantic models as the contract. AskRequest{question: str} / AskResponse{answer: str, sources: list}: FastAPI rejects malformed bodies with 422 before the handler ever runs, so req.question is guaranteed to be a str inside the handler. The OpenAPI schema and the interactive /docs page are generated for free, so frontend never waits on hand-written docs.

3. POST with a JSON body, not GET. The question is arbitrary-length natural language: GET URLs have length limits, need encoding, and end up in access logs; POST body semantics also fit — this is a compute/generate operation, not a static resource fetch. Future parameters (top_k, temperature) become new JSON fields instead of more query params.

4. The graded-honesty prompt held up end to end. A /docs test on "What is RAG?" returned a structured partial answer and explicitly marked what the context did not cover ("Not covered by the context" for the RAG acronym expansion) — no hallucination. This is the over-refusal vs. hallucination seesaw balance point found on Day 3, now reproducing stably behind an HTTP interface.

5. A live replay of the Day 2 finding. The three returned sources are ordered by reranker score: the chunk with the best cosine distance (0.375) ranked only third, while the definitional chunk (0.414) ranked first. The cross-encoder reranks by semantic relevance, not vector distance — and the two scores live on different scales, so they must never be read against each other.

Pitfalls (7, including 2 false alarms)

Pit 1 — results["distance"] → KeyError. Chroma's collection.query() returns distances under the key "distances" (plural). Lesson: never write SDK field names from memory; print the keys or check the docs once.

Pit 2 — PROMPT_TEMPLATE.format(question, context) → KeyError: 'context'. The

template uses named placeholders {question} / {context} but was called with positional args. Rule: {} takes positional args, {name} requires name=value. Mixing them explodes.

Pit 3 — genai.GenerativeModel(...) → AttributeError. Old/new SDK mix-up: from google import genai is the new SDK, which has no GenerativeModel (that class belongs to the old google-generativeai package). Lesson: the import statement determines the API surface — when switching SDKs, switch the whole call pattern. New pattern: client = genai.Client() (auto-reads GEMINI_API_KEY from .env) + client.models.generate_content(...).

Pit 4 — the "it didn't run" illusion. The __main__ block called ask_rag(...) but discarded the return value and printed nothing. The program had actually completed: both models loaded 100% in the logs, generate_content was invoked, zero tracebacks — just no screen output. Lesson: read the log evidence before concluding "it didn't run".

Pit 5 — the "truncated sources" false alarm. The debug print sliced s["text"][:120]; the data in sources was complete all along. Lesson: distinguish display truncation from data truncation.

Pit 6 — the answer stopped mid-sentence. First diagnostic: response.candidates[0].finish_reason. MAX_TOKENS = hit the output token cap → raise it via GenerateContentConfig(max_output_tokens=...). STOP = the model ended normally — this time it was stochastic early stopping; a re-run completed fine. Only if it recurs should the prompt gain a "please answer completely" instruction.

Pit 7 — don't read reranker scores as cosine distances. The distance printed in sources is Chroma's cosine distance; the ordering comes from bge's cross-encoder scores. Different scales, not comparable (the Day 2 conclusion) — mixing them produces the false conclusion that "the ranking is wrong".

第四部分：面试题（仿照真实面试题改编）

以下 5 道题仿照网上真实的 FastAPI / ML 部署面试题改编，每道标注出题依据。格式按你的要求：**中文思路**（怎么想、答哪几点）+ **English answer**（完整英文答案，可直接背）。

Q1. Why did you expose /ask as POST with a JSON body instead of GET with a query parameter?

出题依据：仿照 dev.to 上常见的 “Why would you choose FastAPI over Flask or Django for a new microservice?” 一类 API 设计题，以及 GitHub（sutargaurids/machine-learning-from-scratch-30-days）ML 部署面试题 “Why did you use FastAPI in this project?”。

中文思路：答三点：(1) question 是任意长的自然语言，GET 的 URL 有长度限制、需要编码、还会进服务器访问日志；(2) REST 语义：这不是读取静态资源，而是一次“计算 / 生成”操作，POST 语义更对；(3) 扩展性：以后加 top_k、temperature 等参数，JSON body 直接加字段，GET 要一个个拼 query param。

English answer:

Three reasons. First, the input is arbitrary-length natural language. GET URLs have practical length limits, need percent-encoding, and end up in server access logs — none of which you want for a user-typed question.

Second, REST semantics: this endpoint doesn't fetch a static resource, it performs a compute-and-generate operation (retrieval plus LLM inference). POST is the honest verb for that.

Third, extensibility: when I later add parameters like top_k or temperature, they become new JSON fields. With GET I'd be bolting on more query params and re-validating each one by hand — whereas Pydantic validates the whole JSON body in one place.

Q2. Walk me through exactly what happens from the moment a request hits POST /ask until the response returns.

出题依据：仿照 GitHub（mounishsenisetty/llm-from-zero-to-expert）部署面试题 “What are the components of deploying a model as a service?”——考察“把部署讲成一个完整过程”的能力，而不是只背定义。

中文思路：按链路一步步走：uvicorn（ASGI）收到请求 → Starlette 路由到 handler → Pydantic 校验 AskRequest（失败直接 422，handler 不执行）→ 调用 ask_rag → Chroma 查 top-10 → bge 对 10 个 pair 打分取 top-3 → 拼 graded-honesty prompt → Gemini 生成 → 组 AskResponse → JSON 返回。主动点出最慢的两段：本地 rerank（CPU）和 LLM 生成（网络 + 配额）——这能把话题引向 Q3/Q5 的深度追问，掌握主动权。

English answer:

Uvicorn, as the ASGI server, accepts the connection and hands the request to Starlette, which routes it to my handler based on the path and method. Before the handler runs, FastAPI validates the JSON body against the AskRequest Pydantic model — a missing or mistyped field returns 422 immediately, so inside the handler req.question is guaranteed to be a string.

The handler calls ask_rag(question): Chroma returns the top-10 chunks by cosine distance, the bge cross-encoder scores the 10 question-chunk pairs and keeps the top 3, those chunks are assembled into the graded-honesty prompt, and Gemini generates the answer. The handler then packs everything into AskResponse and FastAPI serializes it to JSON.

Latency-wise, the two expensive stages are the local reranker scoring on CPU and the LLM generation call, which is network-bound and subject to API quota. Retrieval itself is milliseconds against the persisted Chroma index.

Q3. Your handler runs embedding lookup, reranking, and an LLM call synchronously inside the request. Ten users ask questions at the same time — what happens, and how would you improve it?

出题依据：仿照 GitHub（azam123/engineering-interview-prep）FastAPI 并发面试题 “A FastAPI endpoint performs a CPU-heavy operation and latency of unrelated requests suddenly increases. Why?”（原题是 PDF 解析阻塞 event loop；这里换成 RAG 链路的同步阻塞 + 免费配额墙，更贴她的项目）。

中文思路：先讲现状：def（非 async）跑在线程池，不会卡 event loop，但 10 个并发 = 10 个线程各跑约 8 秒；真正的墙是 Gemini 免费配额（约 5 req/min），第 6 个请求就 429。改进三层：(1) I/O 等待 async 化（Chroma 查询、Gemini 调用）释放线程；(2) rerank 是 CPU 密集，扔进程池，别占请求线程；(3) 长期：限流 + 相同问题缓存 + 队列削峰。注意：不要一上来就说“加机器”，先证明瓶颈在哪。

English answer:

Today the handler is a plain def, so it runs in a threadpool — it won't block the event loop, but ten concurrent users mean ten threads each spending ~8 seconds, mostly waiting on the reranker and the LLM call. The harder wall is the Gemini free-tier quota, roughly 5 requests per minute: the 6th concurrent request gets a 429, which my current code doesn't even handle.

I'd improve it in layers. First, make the I/O-bound waits async — the Chroma query and the Gemini call — so threads are freed while waiting. Second, the reranker is CPU-bound; move it to a process pool so it doesn't saturate request threads. Third, for production: rate limiting, caching identical questions, and a queue in front of the LLM call to smooth bursts.

And I'd measure first: profile which stage dominates before scaling anything, because adding machines to a quota-limited LLM call buys nothing.

Q4. You load the embedding model and the reranker at module import time. What are the trade-offs of this choice?

出题依据：仿照 GitHub（mounishsenisetty/llm-from-zero-to-expert）“Why is a naive FastAPI deployment inefficient for LLMs?” 一类部署权衡题——考察“每个设计决策都有代价”的思维。

中文思路：好处：请求零加载延迟，典型的 build-slow / query-fast。代价有三：(1) 启动慢（两个模型加载几十秒），k8s 的 readiness probe 可能在就绪前就杀掉容器；(2) import 时加载失败 = 服务根本起不来，模型文件损坏会拖死整个 API；(3) 多 worker（如 gunicorn -w 4）= 内存里躺 4 份模型，显存/内存翻倍。生产做法：lifespan 事件里加载 + /health 在就绪后才返回 200；或把模型服务拆成独立 sidecar，API 保持无状态。

English answer:

The benefit is zero per-request loading latency — the classic build-slow / query-fast split. Every request pays only for retrieval, reranking, and generation.

The costs are threefold. First, slow startup: loading two models takes tens of seconds, and a Kubernetes readiness probe can kill the container before it's ready if the probe isn't tuned. Second, import-time failure is fatal — a corrupted model file takes down the entire API instead of degrading one endpoint. Third, with multiple workers, say gunicorn -w 4, you get four copies of the models resident in memory.

For production I'd move loading into the lifespan event and only return 200 on /health once models are ready, so orchestrators route traffic correctly. At larger scale, I'd split model serving into a separate sidecar or service so the API layer stays stateless and scales independently.

Q5. Each /ask call takes about 8 seconds end to end. Your PM wants p95 latency under 2 seconds at 10K queries a day. What's your plan?

出题依据：仿照 GitHub（mounishsenisetty/llm-from-zero-to-expert）“How do you scale a model service horizontally?”——考察先算量、再拆延迟、最后谈扩展的顺序感。

中文思路：先算量：10K/day ≈ 0.12
qps，吞吐根本不是瓶颈，瓶颈是单请求延迟——先讲清这点很加分，说明你不会无脑“加机器”。再拆延迟：bge 给 10 个 pair 打分（本地 CPU）+ Gemini 生成是大头，检索只是毫秒级。手段：(1) 缓存：完全相同的问题直接命中（语义缓存是进阶版）；(2) 降 top-k 或换更小的 reranker，用精度换延迟；(3) 流式返回（streaming），首 token 更快，体感延迟大降；(4) 真要扛量：无状态多副本 + 负载均衡横向扩展——但那是吞吐方案，不是延迟方案，不要混。

English answer:
First, the arithmetic: 10K queries a day is about 0.12 queries per second on average. Throughput is not the bottleneck at all — single-request latency is. Naming that distinction up front matters, because horizontal scaling fixes throughput, not latency.

Decomposing the 8 seconds: retrieval against Chroma is milliseconds; the bulk is the bge cross-encoder scoring 10 pairs on CPU plus the Gemini generation round trip.

My plan: one, cache — identical questions hit the cache and skip the whole pipeline (semantic caching is the advanced version). Two, trade accuracy for latency — a smaller reranker or fewer candidates rescored. Three, stream the response so time-to-first-token drops dramatically even if total time is similar. Four, if volume ever does become the problem, stateless replicas behind a load balancer — but that's a throughput answer, and I'd keep it separate from the latency work.

出题依据链接（可自行延伸阅读）：

- [dev.to] FastAPI in production: “Why would you choose FastAPI over Flask or Django?” 的强/弱答案对比 (http://dev.to/techamit95ch/fastapi-in-production-1-fastapi-in-one-hour-529o)
- [GitHub] azam123/engineering-interview-prep：FastAPI 生产并发问答 (https://github.com/azam123/engineering-interview-prep/blob/HEAD/34-python-fastapi-production-concurrency-qa.md)

- [GitHub] mounishsenisetty/llm-from-zero-to-expert：模型部署面试题 (https://github.com/mounishsenisetty/llm-from-zero-to-expert/blob/HEAD/curriculum/28-deployment/README.md)
- [GitHub] sutargaurids/machine-learning-from-scratch-30-days：Day27 “Why did you use FastAPI in this project?” (https://github.com/sutargaurids/machine-learning-from-scratch-30-days/blob/HEAD/Day27/interview_questions.md)